RAPPORT DE PROJET ACADEMIQUE

Pipeline DevOps Complet — CI/CD de bout en bout

Réalisé par : M'BOH Mohamed **Formation** : Informatique & Cybersécurité — Groupe ISM Dakar ǁ PST&B Paris **Date** : Mai 2026

---

1. Introduction

Ce projet a pour objectif la mise en place d'une chaîne DevOps complète autour d'une application web de gestion de tâches (Task Manager). L'application est volontairement simple sur le plan fonctionnel — ajout, affichage et suppression de tâches — car l'enjeu principal réside dans la construction et l'automatisation de l'ensemble de la pipeline CI/CD.

L'objectif pédagogique est de maîtriser le cycle complet, de la gestion du code source jusqu'à la supervision en production :

- Gestion collaborative du code avec Git & GitHub
- Conteneurisation de l'application avec Docker
- Automatisation du pipeline CI/CD avec Jenkins
- Analyse statique de la qualité du code avec SonarQube
- Supervision de l'infrastructure avec Prometheus et Grafana

---

2. Stack Technique

L'ensemble du projet repose sur les technologies suivantes, organisées par couche fonctionnelle :

Couche	Technologie
Frontend	HTML, CSS, JavaScript
Backend	Node.js, Express.js
Base de données	SQLite
Versioning	Git & GitHub
CI/CD	Jenkins (Pipeline Déclaratif)
Conteneurisation	Docker & DockerHub
Qualité du code	SonarQube
Supervision	Prometheus & Grafana
Environnement	WSL2 — Ubuntu 22.04 LTS

3. Architecture de la Chaîne DevOps

La chaîne DevOps mise en place suit le schéma suivant, du commit du développeur jusqu'à la supervision en production :

```
Code (Git/GitHub)  →  Jenkins (Orchestration)  →  Docker (Image)  →
DockerHub  →  Prometheus/Grafana
```

Le pipeline Jenkins enchaîne les étapes suivantes :

- Checkout SCM — récupération du code depuis GitHub
- Install — installation des dépendances (npm install)
- Compile — vérification syntaxique du code Node.js
- Test — tests fonctionnels de la base de données
- SonarQube Analysis — analyse statique de la qualité du code
- Docker Build — construction de l'image Docker
- Docker Push — publication de l'image sur DockerHub

4. Git & Collaboration

4.1 Structure du Repository

```
projet_devops/
├── backend/          # API REST Node.js
├── frontend/         # Interface HTML/CSS/JS
├── package.json      # Dépendances Node.js
├── .gitignore
├── README.md
├── Dockerfile        # branche individuelle uniquement
└── Jenkinsfile       # branche individuelle uniquement
```

4.2 Workflow Git

Branche	Rôle
main	Code stable fusionné et validé
Mohamed-MBOH-BRANCHE	Branche individuelle de développement
khadim-mbodj-BRANCHE	Branche du collaborateur

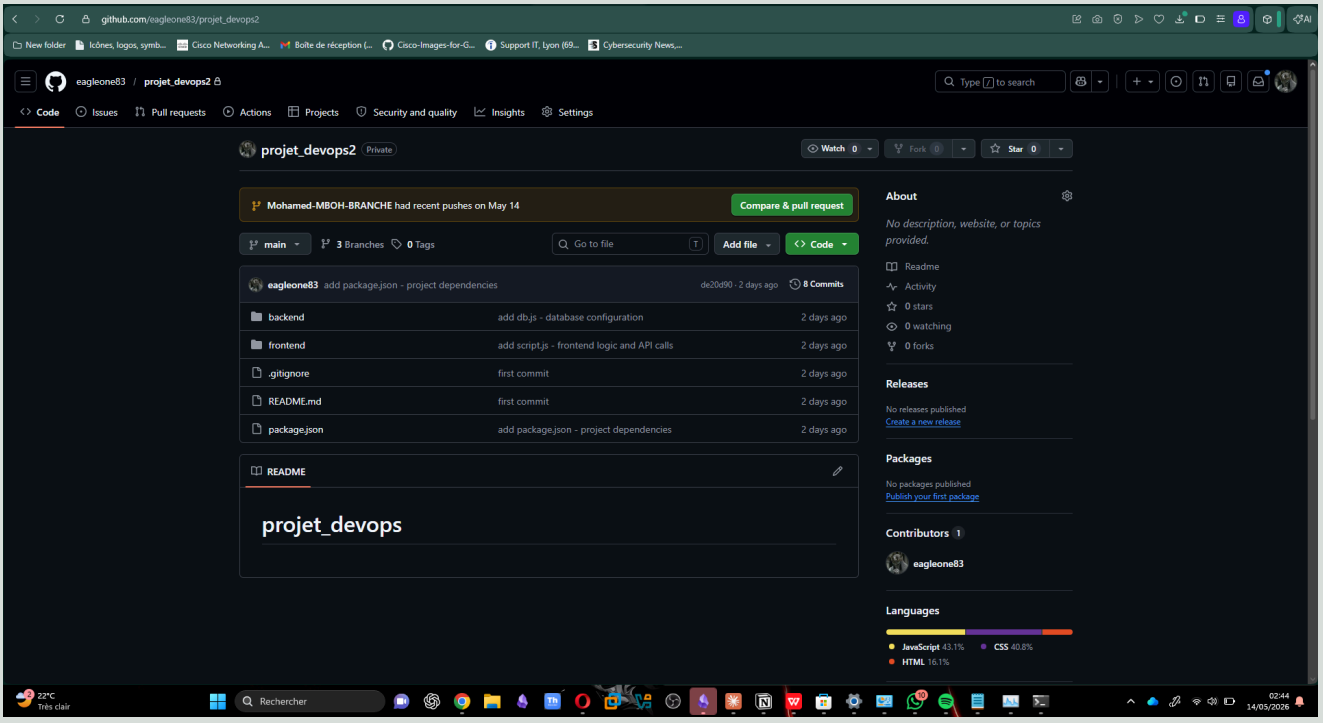
4.3 Commandes Git Utilisées

```
# Cloner le repository
git clone https://github.com/user/projet_devops.git

# Créer et basculer sur sa branche
git checkout -b Mohamed-MBOH-BRANCHE
```

```
# Commiter et pousser
git add .
git commit -m "description du commit"
git push origin Mohamed-MBOH-BRANCHE

# Fusionner dans main
git checkout main
git merge Mohamed-MBOH-BRANCHE
git push origin main
```



---

5. Docker & Conteneurisation

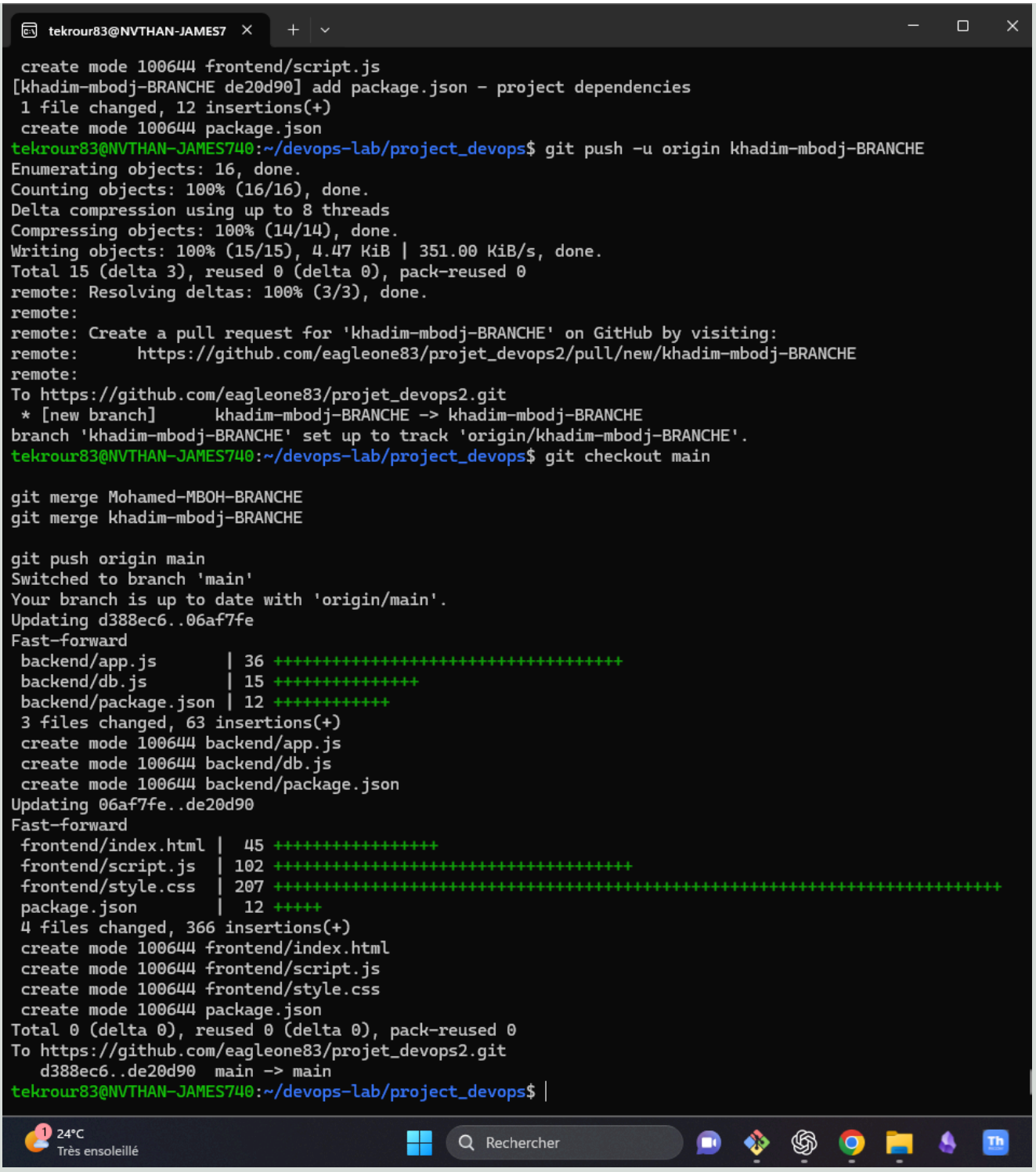
La conteneurisation de l'application avec Docker garantit que celle-ci s'exécute de manière identique dans tous les environnements, éliminant le problème classique du "ça marche sur ma machine".

5.1 Commandes Docker

```
# Construction de l'image
docker build -t user/devops-notes-app:latest .

# Test en local
docker run -p 3000:3000 user/devops-notes-app:latest

# Publication sur DockerHub
echo $TOKEN | docker login -u user --password-stdin
docker push user/devops-notes-app:latest
```


tekrour83@NVTHAN-JAMES7
create mode 100644 frontend/script.js
[khadim-mbodj-BRANCHE de20d90] add package.json - project dependencies
1 file changed, 12 insertions(+)
create mode 100644 package.json
tekrour83@NVTHAN-JAMES740:~/devops-lab/project_devops$ git push -u origin khadim-mbodj-BRANCHE
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (15/15), 4.47 KiB | 351.00 KiB/s, done.
Total 15 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
remote:
remote: Create a pull request for 'khadim-mbodj-BRANCHE' on GitHub by visiting:
remote: https://github.com/eagleone83/projet_devops2/pull/new/khadim-mbodj-BRANCHE
remote:
To https://github.com/eagleone83/projet_devops2.git
* [new branch] khadim-mbodj-BRANCHE -> khadim-mbodj-BRANCHE
branch 'khadim-mbodj-BRANCHE' set up to track 'origin/khadim-mbodj-BRANCHE'.
tekrour83@NVTHAN-JAMES740:~/devops-lab/project_devops$ git checkout main
git merge Mohamed-MBOH-BRANCHE
git merge khadim-mbodj-BRANCHE
git push origin main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
Updating d388ec6..06af7fe
Fast-forward
backend/app.js | 36 ++++++++++++++++++++++++++++++++++++
backend/db.js | 15 +++++++++++++++
backend/package.json | 12 ++++++++++++
3 files changed, 63 insertions(+)
create mode 100644 backend/app.js
create mode 100644 backend/db.js
create mode 100644 backend/package.json
Updating 06af7fe..de20d90
Fast-forward
frontend/index.html | 45 +++++++++++++++++
frontend/script.js | 102 ++++++++++++++++++++++++++++++++++++++
frontend/style.css | 207 ++++++++++++++++++++++++++++++++++++++

Paramètre	Valeur
Script	Jenkinsfile versionné dans le repo

6.2 Jenkinsfile Complet

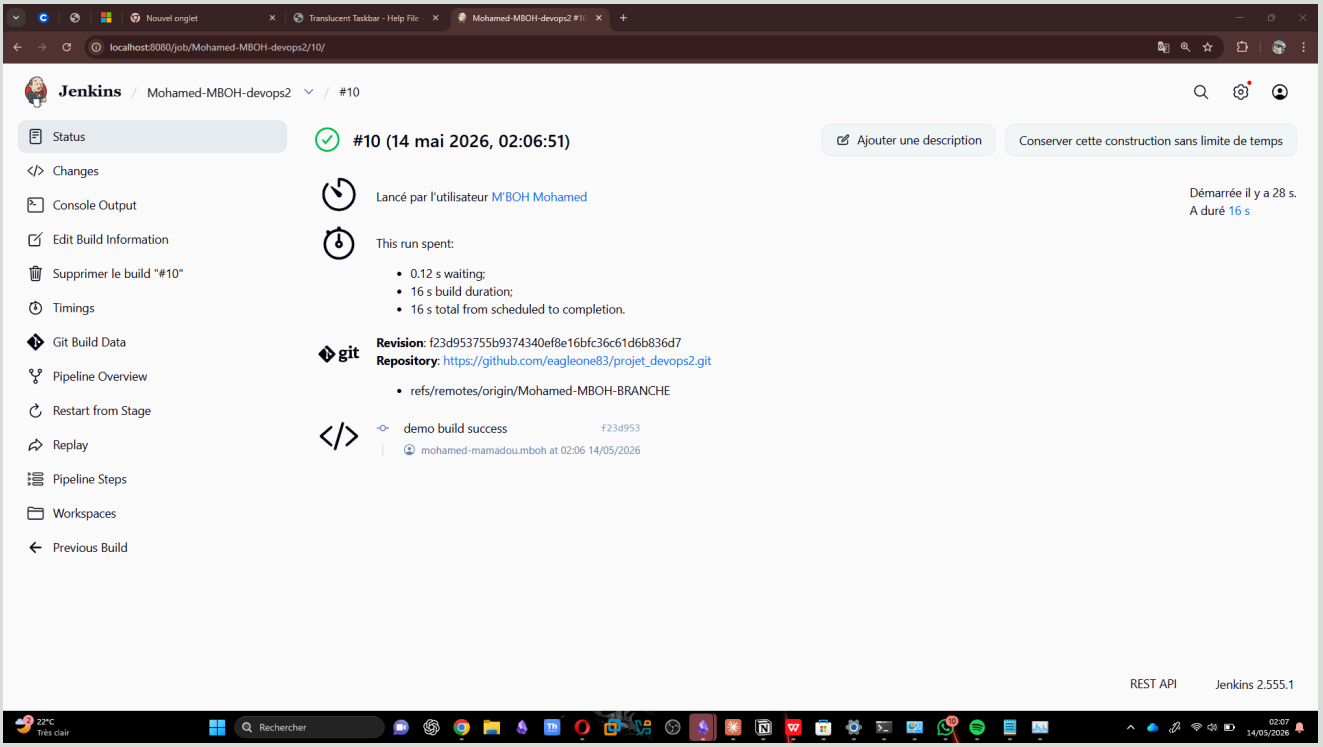
```
pipeline {
    agent any

    environment {
        DOCKERHUB_USER = 'user'
        IMAGE_NAME     = 'devops-notes-app'
        PATH           = "/usr/local/bin:/usr/bin:/bin:${env.PATH}"
    }

    stages {
        stage('Checkout SCM') {
            steps {
                git branch: 'Mohamed-MBOH-BRANCHE',
                    url: 'https://github.com/user/projet.git'
            }
        }
        stage('Install') {
            steps { sh 'npm install' }
        }
        stage('Compile') {
            steps {
                sh 'node --check backend/app.js'
                sh 'node --check backend/db.js'
            }
        }
        stage('Test') {
            steps {
                sh '''node -e "require('./backend/db.js'); console.log('DB
OK')"'''
            }
        }
        stage('Docker Build') {
            steps {
                sh "docker build -t ${DOCKERHUB_USER}/${IMAGE_NAME}:latest
."
            }
        }
        stage('Docker Push') {
            steps {
                withCredentials([string(credentialsId: 'DOCKERHUB_PASS',
variable: 'DOCKER_TOKEN')]) {
                    sh "echo $DOCKER_TOKEN | docker login -u
${DOCKERHUB_USER} --password-stdin"
                    sh "docker push ${DOCKERHUB_USER}/${IMAGE_NAME}:latest"
                }
            }
```

```
        }
    }

    post {
        success { echo 'Pipeline termine avec succes' }
        failure { echo 'Pipeline echoue' }
    }
}
```



7. SonarQube — Qualité du Code

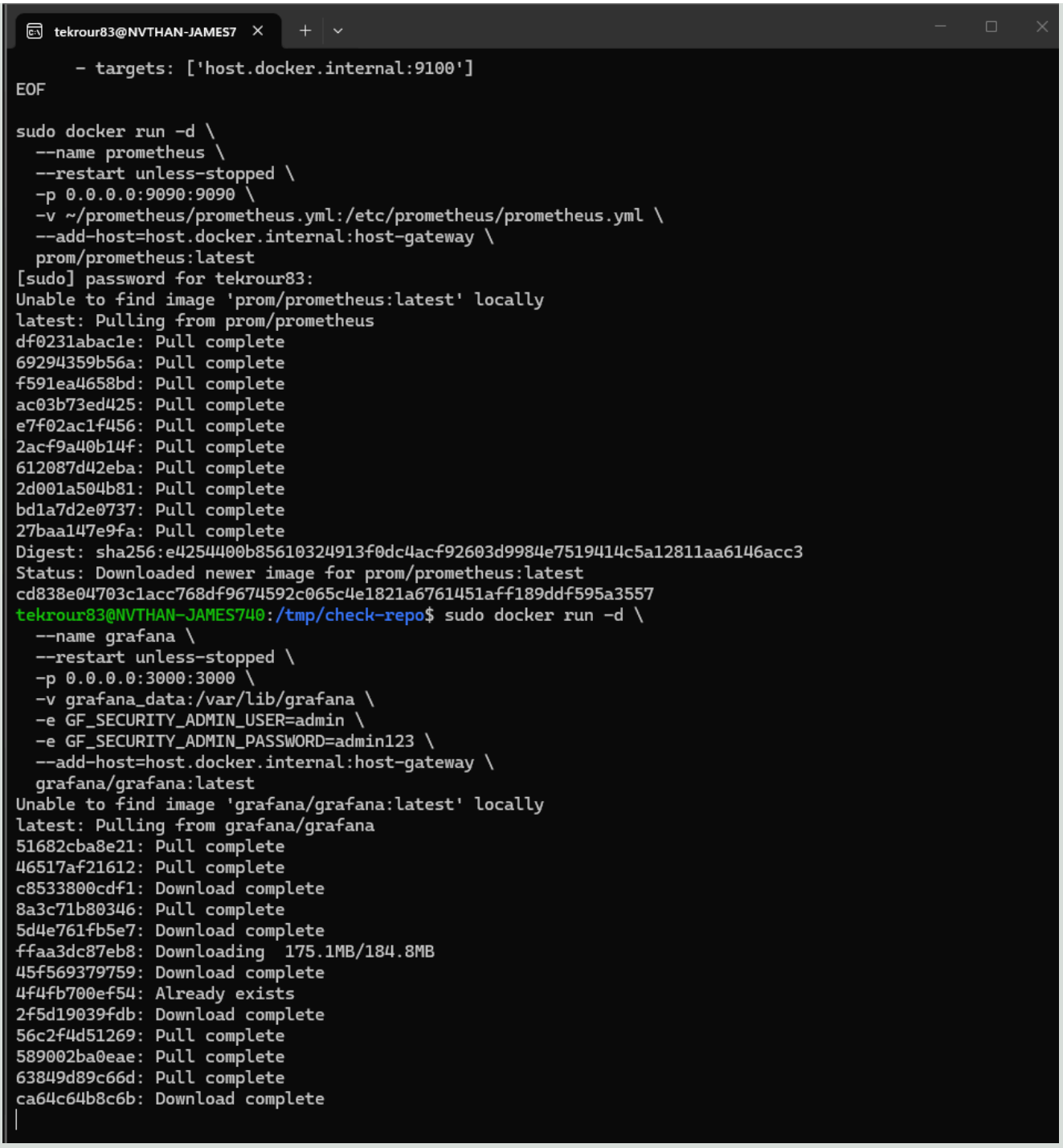
SonarQube est intégré directement dans le pipeline Jenkins. Il effectue une analyse statique automatique du code à chaque build, permettant de détecter bugs, vulnérabilités et mauvaises pratiques avant tout déploiement.

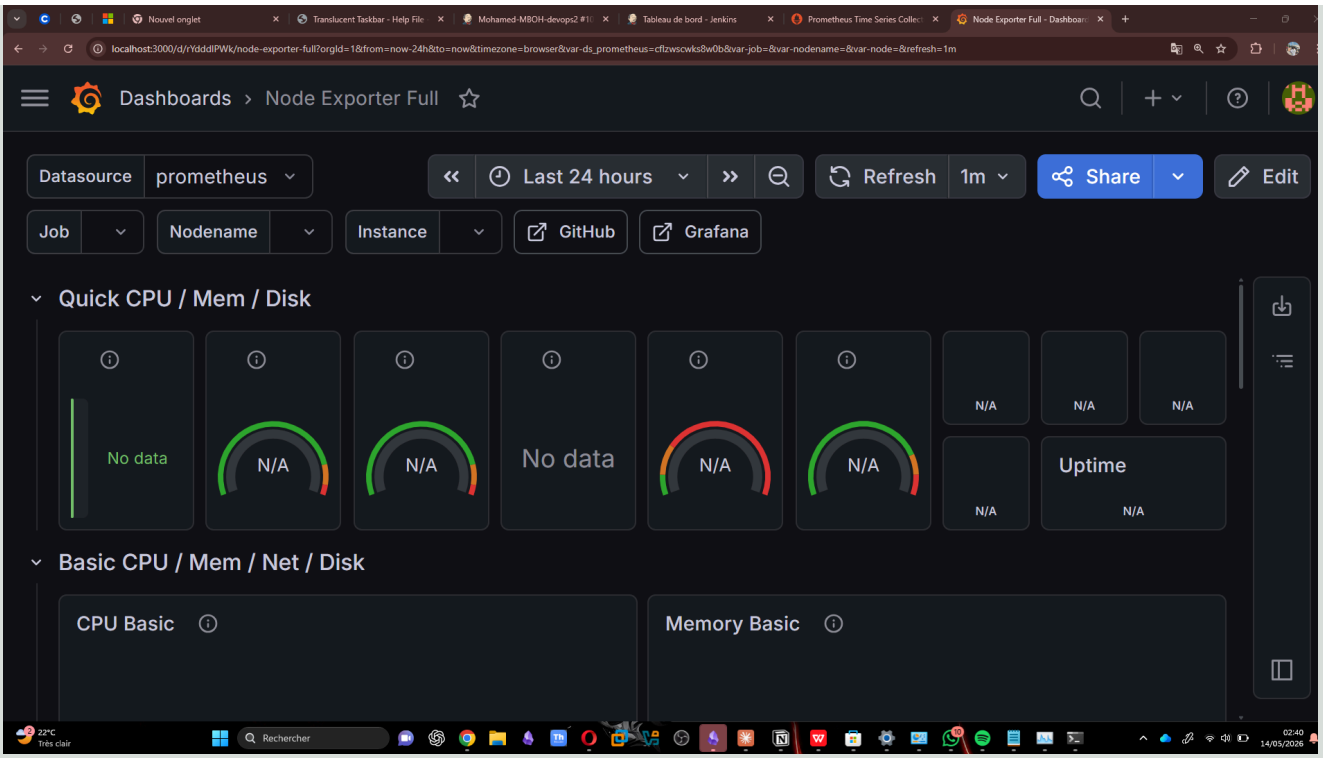
Plugins Jenkins installés

- SonarQube Scanner
- Docker Pipeline

Métriques analysées

Métrique	Description
Bugs	Erreurs détectées dans le code
Vulnerabilities	Failles de sécurité potentielles
Code Smells	Mauvaises pratiques de développement
Coverage	Taux de couverture des tests
Duplications	Pourcentage de code dupliqué


tekrour83@NVTHAN-JAMES7
- targets: ['host.docker.internal:9100']
EOF
sudo docker run -d \
--name prometheus \
--restart unless-stopped \
-p 0.0.0.0:9090:9090 \
-v ~/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml \
--add-host=host.docker.internal:host-gateway \
prom/prometheus:latest
[sudo] password for tekrour83:
Unable to find image 'prom/prometheus:latest' locally
latest: Pulling from prom/prometheus
df0231abac1e: Pull complete
69294359b56a: Pull complete
f591ea4658bd: Pull complete
ac03b73ed425: Pull complete
e7f02ac1f456: Pull complete
2acf9a40b14f: Pull complete
612087d42eba: Pull complete
2d001a504b81: Pull complete
bd1a7d2e0737: Pull complete
27baa147e9fa: Pull complete
Digest: sha256:e4254400b85610324913f0dc4acf92603d9984e7519414c5a12811aa6146acc3
Status: Downloaded newer image for prom/prometheus:latest
cd838e04703c1acc768df9674592c065c4e1821a6761451aff189ddf595a3557
tekrour83@NVTHAN-JAMES740:/tmp/check-repo$ sudo docker run -d \
--name grafana \
--restart unless-stopped \
-p 0.0.0.0:3000:3000 \
-v grafana_data:/var/lib/grafana \
-e GF_SECURITY_ADMIN_USER=admin \
-e GF_SECURITY_ADMIN_PASSWORD=admin123 \
--add-host=host.docker.internal:host-gateway \
grafana/grafana:latest
Unable to find image 'grafana/grafana:latest' locally
latest: Pulling from grafana/grafana
51682cba8e21: Pull complete
46517af21612: Pull complete
c8533800cdf1: Download complete
8a3c71b80346: Pull complete
5d4e761fb5e7: Download complete
ffaa3dc87eb8: Downloading 175.1MB/184.8MB
45f569379759: Download complete
4f4fb700ef54: Already exists
2f5d19039fdb: Download complete
56c2f4d51269: Pull complete
589002ba0eae: Pull complete
63849d89c66d: Pull complete
ca64c64b8c6b: Download complete



Capture — Dashboard SonarQube : résultats de l'analyse →

---

8. Prometheus & Grafana — Supervision

La supervision repose sur le duo Prometheus/Grafana. Prometheus collecte les métriques en temps réel et Grafana les visualise sous forme de tableaux de bord interactifs.

8.1 Configuration Prometheus

```yaml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'jenkins'
    metrics_path: /prometheus
    static_configs:
      - targets: ['host.docker.internal:8080']
  - job_name: 'node'
    static_configs:
      - targets: ['host.docker.internal:9100']
```

8.2 Dashboards Grafana

Dashboard	ID	Description
Node Exporter Full	1860	CPU, RAM, disque, réseau

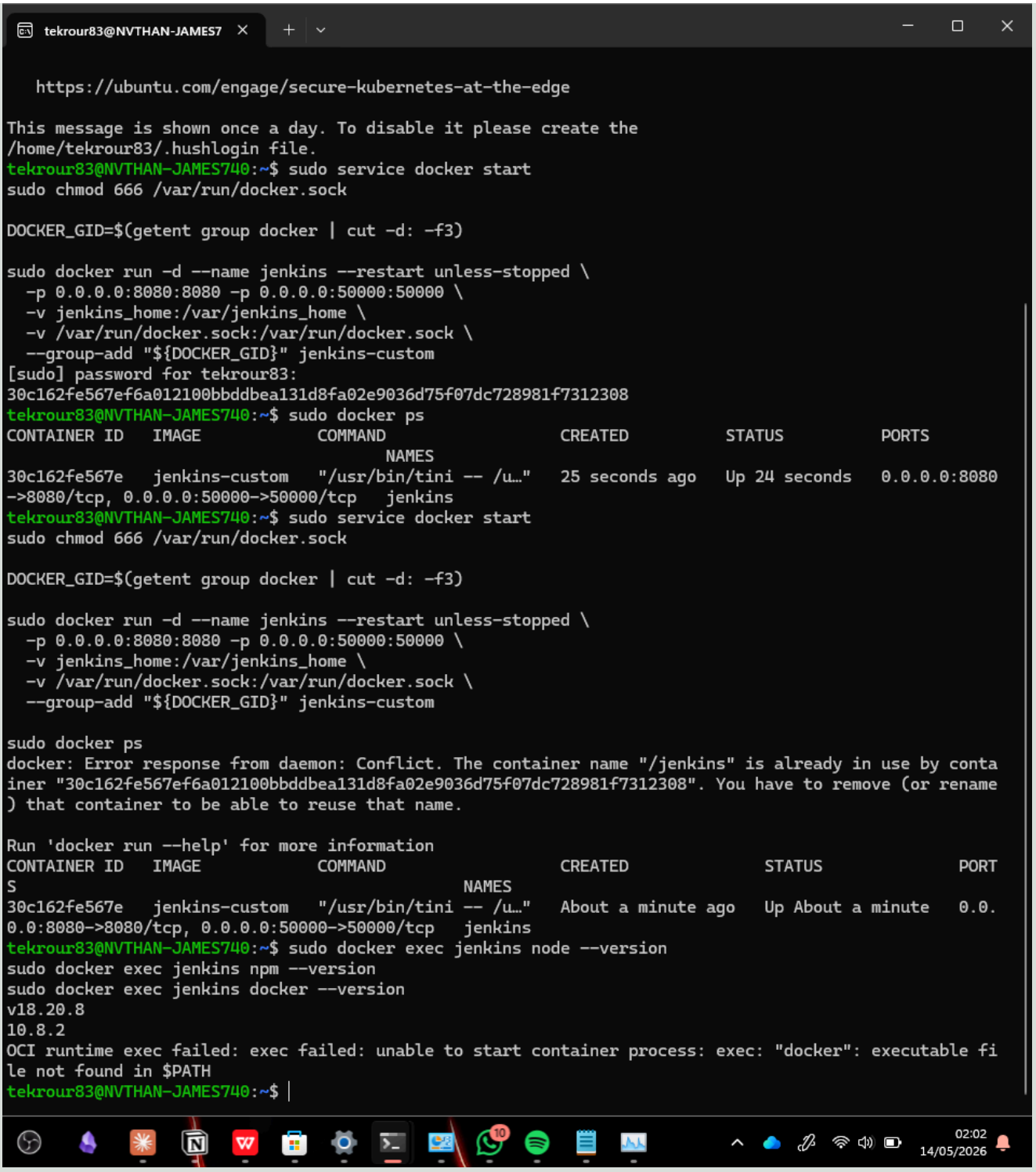
Dashboard	ID	Description
Jenkins Performance	9964	Métriques des builds Jenkins

8.3 Accès aux Services

Service	URL
Jenkins	http://localhost:8080
SonarQube	http://localhost:9000
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 (admin / admin123)

---

9. Vérification Finale



10. Difficultés Rencontrées & Solutions

Problème	Cause	Solution
npm introuvable dans Jenkins	PATH incomplet dans le container	Ajout de PATH dans environment du Jenkinsfile
package.json absent de la branche	Fichier uniquement dans main	git checkout origin/main -- package.json

Problème	Cause	Solution
docker CLI introuvable dans Jenkins	Image de base sans Docker CLI	Rebuild jenkins-custom avec docker-ce-cli
Token GitHub invalide	Token expiré	Génération d'un nouveau Personal Access Token
Credentials non appliqués	Menu laissé sur None	Sélection manuelle de github-creds

---

11. Conclusion

Ce projet a permis de mettre en œuvre une chaîne DevOps complète et fonctionnelle, couvrant l'intégralité du cycle de vie logiciel. Les compétences acquises sont directement applicables en contexte professionnel : gestion Git, conteneurisation Docker, pipeline CI/CD Jenkins, analyse qualité SonarQube et supervision Prometheus/Grafana.

Ce projet illustre concrètement comment les pratiques DevOps permettent de réduire le Time To Market, d'améliorer la qualité du code livré et de maintenir une visibilité permanente sur l'état de l'infrastructure.

Annexe — Speech de Publication

Publication académique — Projet DevOps II Groupe ISM Dakar

Dans le cadre de mon cursus en Informatique & Cybersécurité, j'ai eu l'opportunité de mettre en œuvre une chaîne DevOps complète autour d'une application web de gestion de tâches. Ce projet m'a permis de passer de la théorie à la pratique sur l'ensemble du pipeline CI/CD.

De la gestion des branches Git jusqu'au déploiement automatisé d'images Docker, en passant par Jenkins pour l'orchestration, SonarQube pour la qualité du code et Prometheus/Grafana pour la supervision — chaque outil a été intégré dans un environnement réel sous WSL2/Ubuntu.

Les obstacles rencontrés — configuration de l'environnement, intégration Docker dans Jenkins, gestion des credentials — ont été autant d'opportunités d'apprentissage concret, bien plus formateurs que la théorie seule.

Ce travail s'inscrit dans ma trajectoire vers les métiers de la cybersécurité et de l'administration système, avec une orientation progressive vers la Gouvernance, le Risque et la Conformité (GRC).

#DevOps #CI_CD #Jenkins #Docker #SonarQube #Prometheus #Grafana #Git #Cybersécurité #ProjetAcadémique

---

M'BOH Mohamed — Groupe ISM Dakar / PST&B Paris — Mai 2026